Real Time Operating Systems and Design

Term Project

Dr. Murat YILMAZ

- TRAITORS-

230208048 - Gökçe Hilal Büyük

230208045 – Demir Ege Ortaç

230201010 – Emre Ağan

Project Repository:

https://github.com/demiregeortac666/RTOS_Project.git

Case Study 1: Scheduling and Starvation (20 pt)

1.a (Setup):

In Priority Scheduling, the CPU always runs the task with the highest priority, in this case Task H is always executed as long as it is in the Ready state. Lower priority tasks like Task M and Task L in this case only receive CPU time once Task H has completed or entered a blocked state. While this method responds best to RTOS with time-sensitive requirements it creates an environment where lower priority tasks can be ignored if the high-priority task has a long execution time.

Round Robin Scheduling prioritizes fairness and time-sharing. The CPU allocates a fixed time slice to each task, regardless of their priority levels. This prevents any single task from running all the time. In this case Task M and Task L get regular intervals of execution even when Task H is active. While this eliminates the risk of starvation, it maybe inefficient for critical tasks because the CPU is never allowed to run a task to completion. Instead, CPU is forced to switch contexts frequently, giving each task an equal amount of time to run in each interval.

1.b (The Problem):

When the execution time of Task H is increased, we observe a serious bottleneck in the Preemptive Priority Scheduling logic, where CPU is strictly given to the task with highest priority when that task is in the Ready state.

As a result, Task H which is the task with the highest priority monopolizes the processor for the entire duration of its period, never giving the scheduler a chance to context switch to the lowest priority task which is Task L. This results in Task L being permanenatly stuck in the Ready state, unable to transition into the Running state. This is called Starvation and in our Scheduling Demo it is shown by the red X marks.

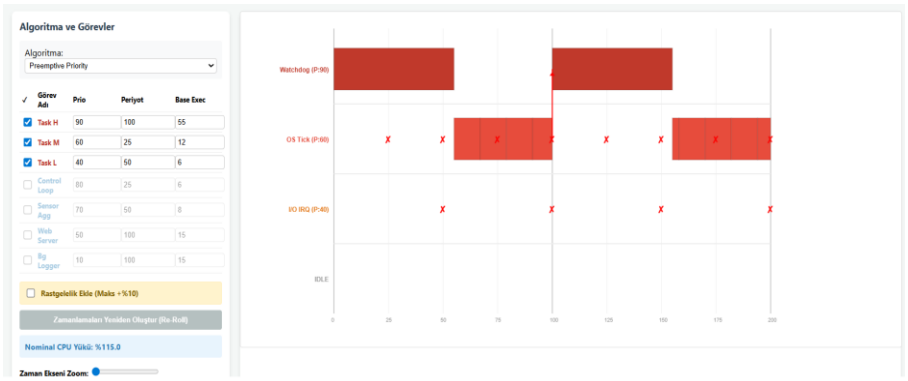

Image 1.1: Simulation Showing How Task L Experiences Starvation

1.c (The Fix):

To resolve the starvation issue, we implement an Aging algorithm within the simulator's scheduling loop. With the aging algorithm we implemented, a task's priority increases based on its time spent in the Ready state. Therefore lower priority tasks like Task L do not wait indefinitely. As we can see in the graph, Task L's priority eventually increases enough to surpass the higher priority tasks like Task H and Task M so, Task L can finally exacute and therefore escape starvation this creates a preemption moment.

Image 1.2: Simulation Where an Aging Algorithm is Implemented

Case Study 2: Producer-Consumer and CPU Idle Time (10 pt)

2.a (Setup):

In order to test dropping or blocking when handling a resource overflow, a fast Producer with a throughput of 100ms and a slow Consumer with a throughput of 500ms were used to send messages to a Message Queue which had a capacity of 5 messages. The fast Producer was able to

fill the capacity of the Message Queue within a few items, thus providing a synchronization bottleneck where the tests for dropping and/or blocking would take place.

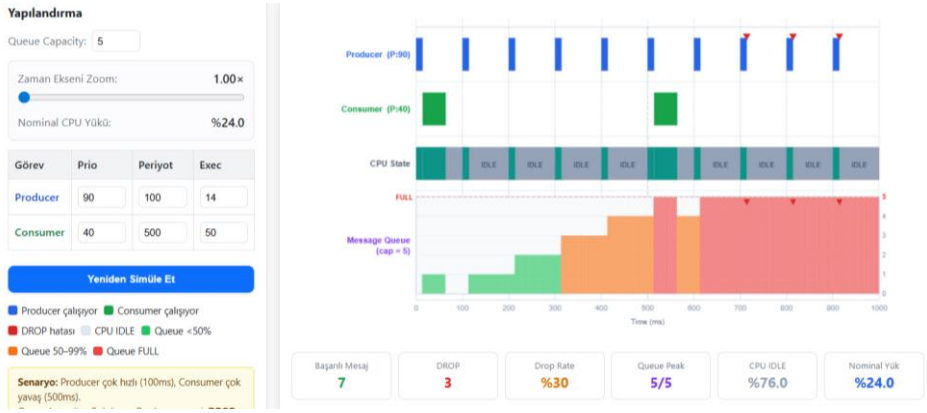

Image 2.1: Simulation Showing Producer-Consumer Bottleneck and Data Drop Errors

2.b (The Problem):

There is a lot of difference between the CPU load that polling and blocking. Firstly, polling load is always high (i.e. 100% in the case of a busy-wait loop), because the CPU spends its time executing instructions to poll for a condition, which may not be true. This means that, even when no data is available, the CPU is wasted in idle cycles. On the other hand, the load that blocking puts on the CPU is very small, as the RTOS will put the task to sleep, allowing other ready tasks to run, or the CPU can go to Idle (low power) until a hardware interrupt or signal wakes it up.

Image 2.2: Synchronization by Producer and Consumer Tasks in Blocked State

2.c (The Fix):

Although blocked mode provides data safety guarantees, for high-end real-time applications it is more effective to support an overwrite mode. The high-priority producer of navigation data (GPS) would otherwise have to wait for the lower-priority consumer to process the data in blocked mode, potentially missing critical timing constraints. In a real-world example, an Unmanned Surface Vehicle (USV) requires a sensing system that can process real-time

navigation data and update the path-planning algorithms for accurate re-mapping and obstacle avoidance. Overwriting in this scenario is safer than blocking. The system processes the latest environmental data instead of old, stale records.

Image 2.3: Simulation Showing Data Overwrite with Ring Buffer Logic

Case Study 3: Priority Inversion (10 pt)

3.a (Setup):

The purpose of this case simulation is to demonstrate the "priority inversion" problem that is seen in Real-Time operating systems. In the simulation, our simulated CPU uses a "preemptive priority-based" scheduling algorithm, meaning that the arriving task with the higher priority. Additionaly, in order to make the system operational, we introduce the CPU 3 tasks, which are as follows: Task H, Task M, Task L. Task H is the highest priority task , Task M is the medium priority task and Task L . Their Priority values are as follows: H = P3, M = P2 and L=P1. The tasks arrive at the system as the following: t=0 (L), t=4 (H), t=5(M). The tasks H and L use the same mutex. Task M does not use mutex, but rather it's a CPU-bound medimum priority task. Task H has a critical section time period of 2 ticks, Task L has a critical section time period of 5 ticks, and M has no critical section time period since it does not use mutex. The simulation works up until 25 ticks. The simulation has 2 mods: "Without Priority Inheritance" and "With Priority Inheritance" which we are going to use to demonstrate the problem of "Priority Inversion" and it's solution we are introducing called "Priority Inheritance".

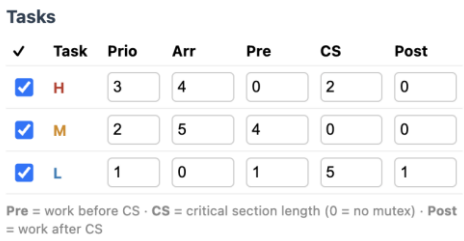

Image 3.1: Simulation Tasks and their corresponding simulation parameters

3.b(The Problem):

In order to demonstrate the "Priority Inversion Problem" we run the simulation in "Without Priority Inheritance" mode. At t=0, Task L arrives (priority 1), at t=1 L acquires mutex, at t=4 Task H arrives (priority 3) and requests mutex but the request is rejected and Task H is blocked because the mutex is held by Task L. At t=5 Task M arrives (priority 2) and starts execution since it does not require mutex and has higher priority than Task L. Task M executes up until t=8 and finished execution at t=8. Then Task L continues its execution since it holds mutex and task H is still blocked up until t=11. At t=11 L releases mutex and Task H becomes unblocked. At t=12 Task H acquires mutex, executes up until t=15. Lastly, as Task L still has an uncompleted Post (work after CS) task section of 1 tick, it continues its execution at t=15 and finished its execution at t=16. The main problem here is, as Task M arrives at t=5, since Task M's priority is higher than L the scheduler gives CPU to Task M. This is a precursor event that will lead to the Priority Inversion issue in the system. Priority Inversion in the system happens because Task H is the task with the NEXT highest priority, however it cannot execute until L releases mutex. Additionally, L cannot release mutex because it is preempted by Task M because Task M has higher priority than Task L. In Figure 3.2 red shaded area shows the duration of time Task H was blocked. Interpreting the simulation timeline screen, we can refer that at the "Without Priority Inheritance" mode, Task H is blocked for 7 ticks and Task H finished at t=15. So we can conclude that, in the system there exists a severe Priority Inversion problem as Task M is completed before Task H. This is a critical problem because in RTOS the delay of a high-priority task can cause deadline miss, this is why Priority Inversion is a critical problem especially in real time systems that use shared resources.

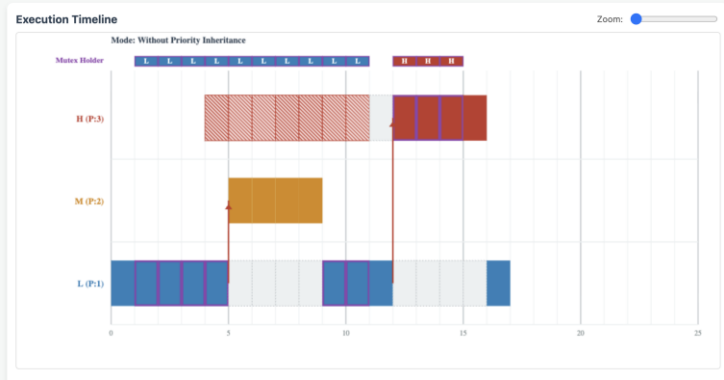

Figure 3.2 : Execution Timeline in the mode "Without Priority Inheritance"

3.c(The Fix):

In order to fix the Priority Inversion Problem. This time, we are going to use the "Priority Inheritance" method by running the simulation in "With Priority Inheritance" mode. As we run the simulation; At t=0 L arrives (priority)1, at t=1 Task L acquires mutex, at t=4 Task H arrives (priority 3) and requests mutex, however it's execution is blocked since mutex is held by Task L. At this point, unlike our previous simulation our system activates Priority Inheritance algorithm , and assigns the priority of the higher task to the task holding the mutex so that the already running Task with lower priority can finish as quickly as possible without any other tasks interfering and the higher priority task can initiate it's execution rapidly after the lower's end of execution. Thus, at t=4 in our simulation Task L inherits the priority of the highest priority task (Task H) which is asking for mutex and Task L's priority becomes from 1 to 3 for a limited time until it's critical section time segment ends and it releases mutex. At t=5, Task M arrives (priority 2), however this time because Task L and Task H has higher priority (3) Task M has to wait until Task L and Task H ends execution. At t=7, Task L completes it's critical section time segment and releases mutex, it's priority gets restored by to it's previous (1) and Task H becomes unblocked. At t=8, Task H acquires mutex, it executes up until t=11, releases mutex and finished its execution. At t=12 Task M (priority 2) starts its execution and executes up until t=15. and finished its execution. Interpreting the simulation timeline screen, we can refer that at

the "With" Priority Inheritance" mode, Task H was blocked for 3 ticks and finished its execution at t=11. In the light of the data we've gathered from both simulation modes, we can conclude that, rather than eliminating the Priority Inversion issue absolutely, Priority Inheritance algorithm is a powerful solution for taking it under control as Task H still has to wait for Task L's mutex, but Task M cannot interfere and increase the waiting duration of Task H.

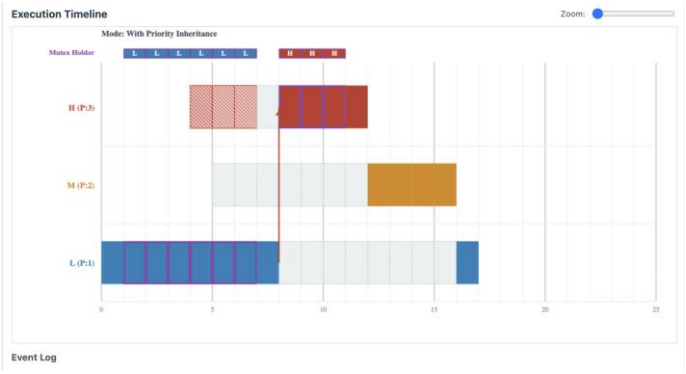

Figure 3.3 : Execution Timeline in the mode "With Priority Inheritance"

4.a (Setup):

The purpose of this case simulation is to demonstrate a possible deadlock situation that might occur between two Tasks and two mutexes. The simulation uses the Round-Robin scheduling algorithm. The length of the simulation is 25 ticks. The simulation has 2 tasks as the following: Task A and Task B, has 2 mutex protected resources as the following: M_Sensor and M_Disk. The temporal order of Task A's mutex request is as folllows: M_Sensor, M_Disk. The temporal order of Task B's mutex request is as folllows: M_Disk, M_Sensor. As the tasks A and B try to request mutexes on reverse order, this situation gives rise to a possible circular wait condition, which can end up in a deadlock. The arrival time of the both tasks are t=0 and the critical section length (CS) for both the tasks are CS=1 tick. In order to demonstrate the Deadlock problem and its possible fix, we developed a dual mode simulation which features the "No Recovery" and "Mutex Timeout" modes.

TASK	ARRIVAL	1ST MUTEX	2ND MUTEX	CS
A	0	M_Sensor	M_Disk	1
B	0	M_Disk	M_Sensor	1

Table 4.1: Task and mutex acquisition order for the deadlock simulation

4.b (Problem):

In order to demonstrate, the Deadlock problem caused by circular wait condition, we run our simulation in "No recovery" mode. In this mode, if Deadlock occurs the system does not interfere with tasks for Deadlock "prevention". Both the Task A and Task B arrive at t=0, however since our CPU has only one core, and scheduler uses Round-Robin scheduling algorithm, CPU selects one task, puts it in the working mode and other stays in ready mode. In our example Task A stars working at t=0 and Task B starts working at t=1. At t=2, Task A acquires the mutex protected resource M_Sensor and at t=3 B acquires mutex protected resource M_Disk, meaning that during the execution of each tasks, no other tasks can acquire the given resources M_Sensor and M_Disc except from the Tasks who already acquired them (Task A and B respectively). At t=6, Task A requests M_Disc, however Task A is blocked because, Task B haven't released the mutex protected resource M_Disc yet. Likewise, at t=7 Task B requests M_Sensor, however Task B is now also blocked because Task A haven't released it yet. Since neither task can continue without mutex held by the other task both tasks remain blocked. As a result, we detect a Deadlock occurring due to Circular Wait at t=7.

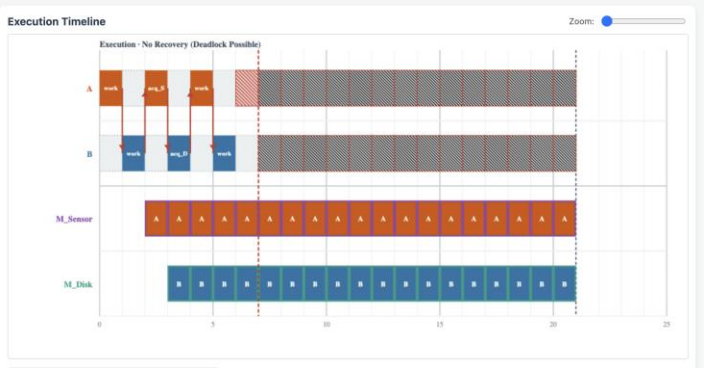

Table 4.2: Deadlock outcome in No Recovery mode

4.c (Fix):

In order to prevent the Deadlock, we are introducing a method called "Mutex Timeout" by running the "Mutex Timeout" mode in our simulation. In the previous "No Recovery" mode, when deadlock occurs the system does not interfere for prevention. However, Mutex Timeout mode does not allow tasks to wait for mutexes forever. If one task cannot acquire mutex for a specific time interval, it will hit into a timeout. A time occurrence, creates the system recovery behaviour. In our simulation the timeout duration is 4 ticks. Thus, if a task cannot acquire the mutex it's waiting for no more than 4 ticks, system triggers the timeout mechanism. When a timeout occurs, the blocked task stops waiting for the mutex, it releases the mutexes it's been holding, transitions to a short backoff/rollback state (1 tick) and prepares for a new mutex request. Hereby, the circular wait condition in the system causing the Deadlock gets eliminated. This works because Task A was holding M_Sensor resource and waiting for M_Disk resource,

Task B was holding M_Disk and waiting for M_Sensor resource. Since both didn't release what they were holding respectively, no one was able to keep moving. Mutex Timeout eliminates this because the the task that was a time out cannot hold its mutex. At t=0 Task A and Task B arrives, at t=2 Task A acquires M_Sensor, at t=3 Task B acquires M_Disc. Up until t=6, both Task A and Task B executes. However, at t=6 Task A requests M_Disc, however since it's held by Task B , Task A is blocked. Likewise, at t=7, Task B requests M_Sensor, however since it's held by Task A, Task B now is also blocked. As a result, we observe a deadlock occurring at t=7. After 4 ticks, which is the mutex timeout duration at t=10, Task A hits a timeout and stops waiting for M_Disc, rollbacks and releases M_Sensor. Followingly, as M_Sensor is now released, Task B is unblocked. At t=11 Task A's backoff ends and it retries critical section and Task A acquires M_Disc. At t=12, Task B requests M_Sensor, however as it's now held by Task A, Task B is blocked. Likewise, at t=14 Task A requests M_Disc, however since it's held by Task B Task A is now also blocked. Thus, at t=14 we detect another deadlock occurring. After 4 ticks, which is the mutex timeout duration, at t=16, Task B hits a timeout and stops waiting for M_Sensor, rollbacks and releases M_Disc. Followingly, as M_Disc is now released, Task A is now unblocked. At t=17, Task B's backoff ends, retries critical section and Task A acquires M_Disc.

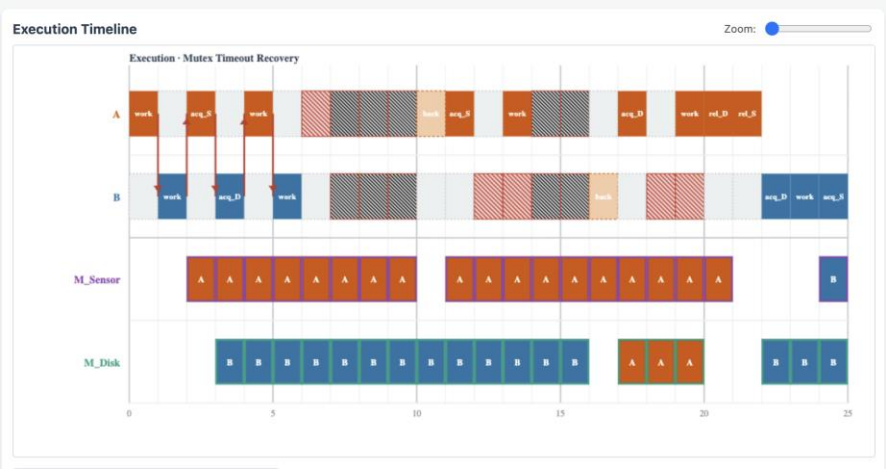

Figure 4.3: Deadlock recovery with Mutex Timeout mechanism

Case Study 5: Full System Integration (25 pt)

5.a (Setup):

This case study implements a simple ADAS (Advanced Driver Assistance System) in a multi-task environment with an RTOS-like structure, allowing you to see how interrupt handling, queue communication, and mutex-based synchronization are managed in real time. The system is composed of five main components:

Component	Type	Period / Exec Time	Role
ISR_Radar	Interrupt Service Routine	Every 20 ms	Reads radar sensor; pushes raw obstacle data to Queue_Radar
Queue_Radar	FIFO Message Queue	Capacity: 5 messages	Decouples ISR from processing task; buffers incoming radar data
Task_Compute	Processing Task	Exec: 8 ms per message	Dequeues radar data; computes risk; acquires Mutex_Brake if threat detected
Mutex_Brake	Binary Mutex	Lock duration: 10 ms	Serialises exclusive access to brake actuator hardware
Task_Log	Periodic Task	Period: 50 ms / Exec: 14 ms	Monitors system state; writes to SD card; waits if Mutex_Brake is held

Table 5.1: System components

The simulation runs for 600 ms, which is long enough to see a few or more ISR cycles, and allows several system interactions to occur. An obstacle probability of 55% is set, which means not every ISR activation will result in a message being added to the queue. The system is preemptive priority-based. ISR_Radar has the highest priority, followed by Task_Compute and Task_Log. This means time critical actions like braking decisions will be executed before less time sensitive actions such as logging.

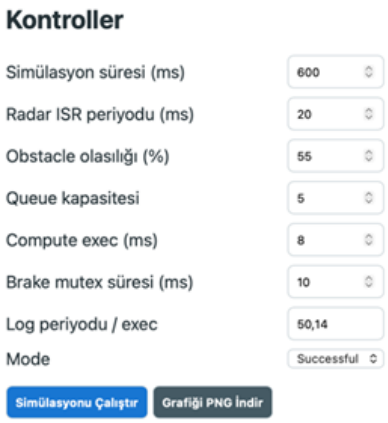

Image 5.2: Settings

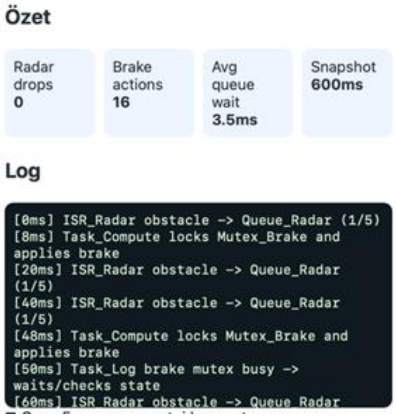

Image 5.3: Log and summary

5.b (Observations)

The simulation ran for 600 ms, you can see on the time line that there are stable interactions between interrupts and tasks. ISR_Radar occurs every 20 ms but the execution time is only 1 ms. This is done on purpose in order to not block the CPU with heavy interrupt code. Messages are only generated when there is an obstacle detected by the sensor, and not in every interrupt cycle. This is more realistic because events, like obstacle detection, do not occur on a regular basis. The Queue_Radar behavior is roughly balanced around a queue depth of 0 to 2. There are brief spikes up to 4 when there are bursts of obstacle detections that get processed by Task_Compute shortly afterwards. In simulations with increased load above the capacity of Queue_Radar, the system correctly drops messages to prevent excessive growth of the queue. This means the system is capable of managing data flow as well as the rate of data consumption.

Image 5.4: Successfull simulation

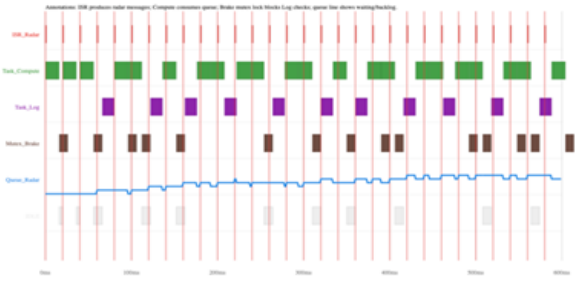

Image 5.5: Overload / Queue waiting

5.c (Final Defense)

I used AI to generate some code, but the generated code had a couple of serious errors. The first error is a Mistake 1 in ISR design. The generated code includes heavy processing logic (in this case, risk evaluation) inside an ISR. Remember, ISR code runs on a CPU that could be locked up by heavy computation. So, the correct code sends the measured data to a queue inside the ISR, and the compute task makes all the decisions in the main thread with minimal latency. Mistake 2 was using the queues to their full extent. In the original code every ISR tick would send a

message. This is far from reality and I made sure that messages are sent only when there is an obstacle detected, simulating a burst of data and not a linear increase. It is also important to make sure that the tasks are consuming the data from the queues at the correct places in the timeline. This ensures a good producer/consumer balance, adheres to the proper RTOS design principles.

6.a (Setup):

This case builds on by I/O priority by introducing network access to a single network interface as a shared I/O resource. Both ISR_Radar (high priority) and Task_Log (low priority) generate packets to be transmitted on this single network interface. To model this, an Ethernet Controller and an Ethernet Driver are added to the system, as well as a Network Stack Task and a TX Queue to manage the outgoing packets. Since both NetworkStack and Filesystem rely on the same core driver and hardware, there is potential for system bottleneck. For the sake of testing and fine tuning the performance under different load scenarios, the priority for the Network Stack Task has been made configurable, allowing for comparison against the Filesystem task.

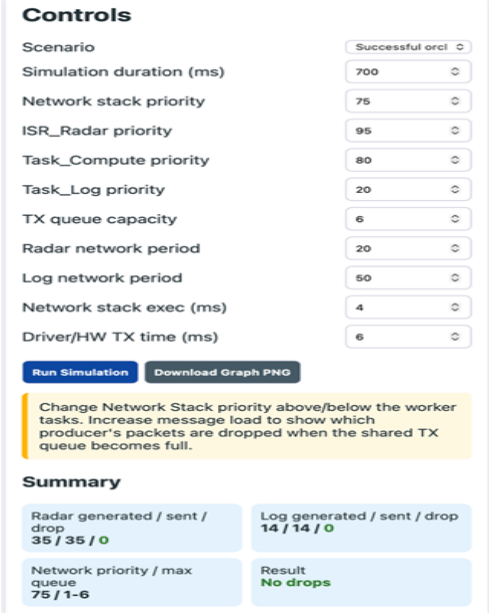

Image 6.1: Settings

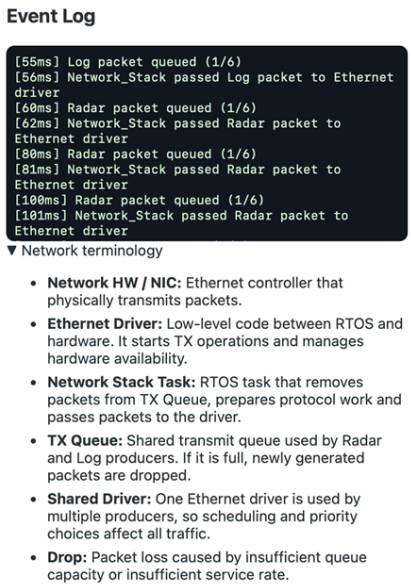

Image 6.2: Log and summary

6.b (Observation):

This case study examines how system behavior affects packet transmission from the TX Queue and Ethernet Driver by the Network Stack Task. Packets are being generated by ISR_Radar and Task_Log and the task is able to process some of them before it is either preempted by other high priority tasks or delayed. As a result, the TX queue level increases and can potentially lead to a Drop if the queue reaches maximum capacity.

Queue dynamics are largely governed by the higher level scheduling and system load rather than simply the generation rate of packets. In a well tuned system packets would be processed on the fly maintaining a very low (pseudo-) queue depth. In this shared driver scenario however the visible depth of the queue is indicative of the real time efficiency of the RTOS in handling contention between high priority interrupt service routines and the lower priority logging routine.

6.c (Final Defense):

Scenario 1: Successful Network I/O

In this scenario, the Network Stack Task priority was set sufficiently high so that packets could be placed into the TX queue in a timely manner and processed by the Ethernet driver. As a result, the queue level remained low and stable.

We are seeing both radar and logging packets on the machine, and no packet drops observed. It appears to be well balanced and the shared driver is being managed correctly.

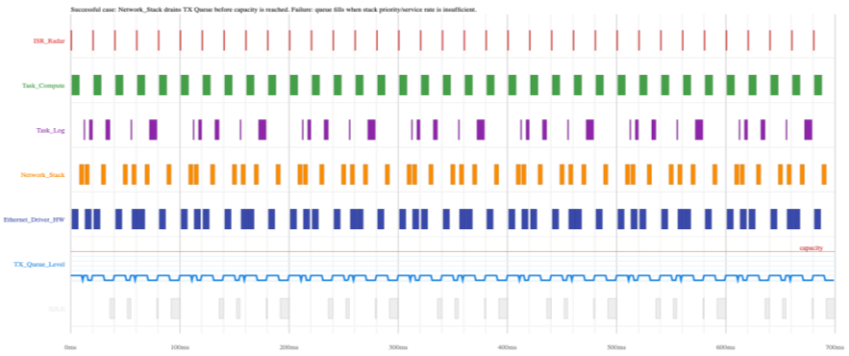

Image 6.3: Successful Network I/O

Scenario 2: Unsuccessful Network I/O (System Orchestration Failure)

In this scenario, the system failure is caused by an imbalanced packet generation and processing. Firstly, the priority given to the Network Stack Task is lower than many other tasks running in the system (such as ISR_Radar), meaning higher-priority tasks always get to use the CPU first, and thus starve the stack task of the cycles it needs to get packets out of the queue. Secondly, even with the stack task running at a reasonable priority, it is possible for the system to exceed capacity, particularly if Network Load is high, and packets are generated faster than the Ethernet Driver can send them.

In this example we can see that the TX Queue is full and therefore packets are Dropped. The same happens in the corresponding increased sample. This is a common issue for network orchestration: proper priority scheduling is not enough, there is also a limit to the workload that can be handled by the hardware, i.e. total bandwidth (in this case throughput).

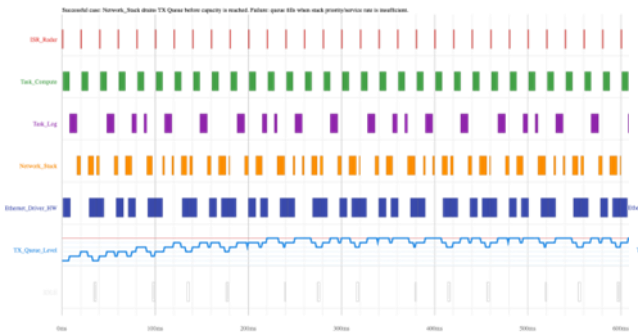

Image 6.4: Low network stack priority

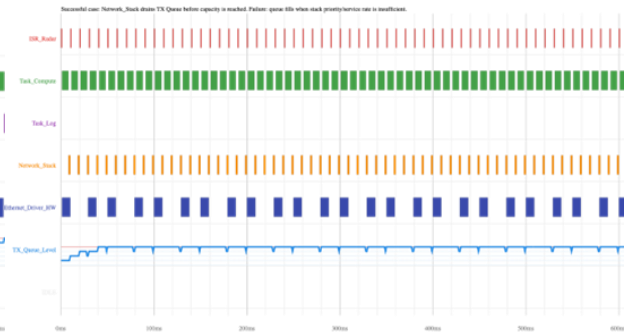

Image 6.5: High network load

AI Use: We used Gemini 3 Flash and Claude to generate the initial HTML frameworks for the simulation demos, and we later used the same models to help us integrate technical fixes as encouraged by the assignment description.